Part 3 — Challenge 2: quantize the storyteller (int8) and measure the cost

Part 2 ended at a ceiling: tokens/sec is ultimately bounded by how fast the model's **weights** can be streamed — and the biggest of them, the classifier, is `32000 × 288 ≈ 9.2 million` numbers, re-read every batch. This challenge attacks that ceiling with the int8 GEMM you built in Part 1: **4× smaller weights, more multiply-adds per unit of hardware** — the technique that makes language models fit and run on real edge devices (LLM.int8, GPTQ, SmoothQuant). Then it asks a question only a storyteller model can answer: **how far can you quantize before the story stops making sense?**

3.1 Why int8 raises the ceiling

The weight roof from Part 2 is a bytes-per-second limit. Quantizing weights from fp32 to int8 cuts the bytes **4×**: the classifier's weights drop from ~37 MB to ~9 MB, the six FFN layers likewise. Fewer bytes to move per token means the streaming roof moves up. On top of that, integer multiply-adds are cheaper than floating-point ones, so the same hardware does more of them per second. You already proved the int8 GEMM correct in Part 1 (`test_gemm_int8.sh`); here you put it to work.

3.2 The golden and the seam

The model engine ships in two forms: the float one you used in Part 2, and an int8 one (`runq`) built on **Q8_0 group quantization** — the exact scheme from Part 1 (`GS` elements per group, int8 values + one fp32 scale, int32 accumulate). `runq` 's own matrix-multiply is your **golden reference**: the int8 story it produces is the target your accelerated version must reproduce.

Quantize the checkpoint once, then run it:

```
cd mini1-gemm/build/llama2.c
python export.py stories15M_q80.bin --version 2 --checkpoint stories15M.bin   # make the
./runq stories15M_q80.bin -t 0.0 -s 42 -i "Once upon a time"                  # int8 stor
```

(If `export.py` needs PyTorch, do that one-time step on a machine that has it and stage the ~15 MB `*_q80.bin` on shared storage.) The plug-in seam mirrors Part 2, one datatype down: an `int8_backend.c` routes `runq` 's quantized matmul to your `student_gemm_int8` , exactly as `slm_backend.c` did for float. You write that small seam file — it is a copy of the float one with the quantized argument types.

3.3 The experiment — "quantize until the story breaks"

This is the deliverable, and the reason a *storyteller* model was the right choice: its correctness is legible to a human. A wrong quantization doesn't throw an error — it writes nonsense. Sweep precision and record **three axes at once**:

Precision	Weight bytes (classifier)	tokens/sec	Story quality (read it!)
fp32 (Part 2)	~37 MB	baseline	perfect — "Lily and the sun"
int8, GS=64	~9 MB	*measure*	still coherent?
int8, GS=32	~9 MB + more scales	*measure*	—
int4 (if you dare)	~4.6 MB	*measure*	where does it fall apart?

Somewhere down this table the model stops writing English. **Find that edge and report it** — the exact bit-width and group size where *"Once upon a time, there was a little girl named Lily..."* degrades into non-sequiturs. That boundary is quantization error made *visible* — a better lesson in numerical precision than any error norm, because you can read it.

3.4 What to report

1. The int8 story next to the fp32 story — do they tell the same tale?
2. **tokens/sec vs precision** and **memory vs precision**, on top of the Part 2 float ceiling. How much did int8 raise the roof, and does your Part 1 int8 kernel deliver it here?
3. Your `student_gemm_int8` matching `runq` 's golden inside the model (correctness before speed — the same discipline as Part 1).
4. One paragraph: the **accuracy / throughput / memory** trade-off you would ship, and why. There is no single right answer — defend yours with your three curves.

Where this leaves you

You have taken a real language model from a plain baseline, found that it is essentially a stack of GEMMs, accelerated those GEMMs with your own kernel, and quantized them to int8 — the full arc of making a model fast on custom hardware. The ceilings you hit along the way (Amdahl's law, the `N = 1` memory wall, the weight-streaming roof, quantization error) are exactly the ones that motivate everything that comes after: fusing operations so data never round-trips, keeping operands moving on-chip, and specializing precision to the task. Your storyteller is the first system where all of it has a number attached.

Checkpoint

- [ ] `runq` generating an int8 story + its tokens/sec.
- [ ] `student_gemm_int8` matching the golden inside the model.
- [ ] The three-axis table filled — including the precision where the story breaks.
- [ ] Your shipped operating point, defended with the curves.